

**BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI**  
**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION SYSTEMS**

**Compiler Construction (CS F363)**

**II Semester 2025-26**

**Compiler Project**

**Coding Details**

**(March 02, 2026)**

Group Number

**12**

**1. Team Members Names and IDs**

ID: 2022B3A71033P

Name: Arjun Neekhra

ID: 2022B4A70596P

Name: Arpita Tomar

ID: 2022B4A70581P

Name: Arvind Annamalai Balasubramanian

ID: 2022B3A70604P

Name: Mehul Srivastava

ID: 2022B2A71101P

Name: S Pranav Kumar

ID: 2022B3A70453P

Name: Tarun G

**2. Mention the names of the Submitted files :**

1 \_\_lexerDef.h\_\_

9 \_\_firstfollow.h\_\_

17 \_\_t1.txt\_\_

2 \_\_lexer.h\_\_

10 \_\_firstfollow.c\_\_

18 \_\_t2.txt\_\_

3 \_\_lexer.c\_\_

11 \_\_parserDef.h\_\_

19 \_\_t3.txt\_\_

4 \_\_grammarDef.h\_\_

12 \_\_parser.h\_\_

20 \_\_t4.txt\_\_

5 \_\_grammar.h\_\_

13 \_\_parserc\_\_

21 \_\_t5.txt\_\_

6 \_\_grammar.c\_\_

14 \_\_driver.c\_\_

22 \_\_t6.txt\_\_

7 \_\_set.h\_\_

15 \_\_makefile\_\_

23 \_\_testcase1.txt\_\_

8 \_\_set.c\_\_

16 \_\_grammar.txt\_\_

24 \_\_coding details.pdf\_\_

**3. Total number of submitted files (including copy the pdf file of this coding details pro forma) : \_\_24\_\_ (All files should be in ONE folder named as Group\_#)**

**4. Have you compressed the folder as specified in the submission guidelines? (yes/no) \_\_YES\_\_**

**5. Lexer Details:**

[A]. Technique used for pattern matching: \_\_ DFA-based rule matching using a hand-coded transition logic (finite-state machine) with the longest-match (maximal munch) principle and limited retract/backtracking where required by the language spec. \_\_

[B]. Keyword Handling Technique: \_\_ keyword lookup is done via search algorithm \_\_

[C]. Hash function description, if used for keyword handling: \_\_ Not used, for a small table, searching algorithms are efficient as well. \_\_

[D]. Have you used twin buffer? (yes/ no) \_\_ yes \_\_

[E]. Error handling and reporting (yes/No): \_\_ yes \_\_

[F]. Describe the errors handled by you \_\_ Our code handles the LEXERR

1. LEXERR\_UNKNOWN\_SYMBOL: unknown characters or invalid starts like \_ not followed by alpha, = alone, etc.

2. LEXERR\_ASSIGN\_INCOMPLETE: <- and <-- treated as lexical error

3. LEXERR\_BAD\_RNUM: malformed real numbers (wrong decimal digit count, bad exponent format, etc.)

4. LEXERR\_BAD\_NEQ: ! not followed by =

5. LEXERR\_BAD\_AND: &, && (only &&& is valid)

6. LEXERR\_BAD\_OR: @, @@ (only @@@ is valid)

7. LEXERR\_TOO\_LONG\_LEXEME: length constraint violations (\_... FUNID length > 30, TK\_ID length not in [2,20])\_\_

[G].Data Structure Description for tokenInfo (in maximum two lines): \_\_\_\_ tokenInfo has 5 field type (storing token type), lineNo, errCode, lexeme (String), value (numerical value of token if it is NUM/RNUM)\_\_\_\_\_

## 6. Parser Details:

[A].High Level Data Structure Description (in maximum three lines each, avoid giving C definitions used):

- i. grammar : \_\_ Grammar is stored as (1) a symbol table (array of Symbol: name, id, kind=Terminal/NonTerminal) and (2) a list of productions, where each production is keyed by an LHS nonterminal and contains multiple RHS alternatives. EPS and \$ are reserved fixed symbols. \_\_\_\_\_
- ii. FIRST and FOLLOW sets \_\_\_\_ FIRST and FOLLOW are maintained as BitSet arrays indexed by symbol-id, i.e., FIRST[X] and FOLLOW[X] are sets of symbol-IDs. \_\_\_\_\_
- iii. parse table \_\_\_\_ Parse table is a 2D matrix of entries (PTEntry) indexed by (NonTerminal row, Terminal column) with mapping arrays rowOfSymbolId and colOfSymbolId.Each cell stores either (prodIdx, altIdx) to pick a production alternative, or special values empty (-1) / SYNCH (-2) used for panic-mode recovery. \_\_\_\_
- iv. parse tree: (Describe the node structure also) \_\_\_\_ Parse tree is an n-ary tree using first-child / next-sibling representation.Each node stores grammar symbol-id, whether it's terminal, a flag if it has an attached token, the tokenInfo (for terminals), and pointers: parent, firstChild, nextSibling. \_\_\_\_\_
- v. Any other (specify and describe) \_\_\_\_ Parser stack: a dynamic stack of (symId, ParseTreeNode\*) items used for predictive parsing and simultaneous parse-tree construction. \_\_\_\_\_

[B].Parse tree

- i. Constructed (yes/no):\_\_\_\_yes\_\_\_\_
- ii. Printing as per the given format (yes/no): \_\_\_\_yes\_\_\_\_
- iii. Describe the order you have adopted for printing the parse tree nodes (in maximum two lines)  
We print the parse tree using an inorder traversal for an n-ary tree : traverse the leftmost child, then print the current node, then traverse remaining children left-to-right. \_\_\_\_\_

[C].Grammar and Computation of First and Follow Sets

- i. Data structure for original grammar rules \_\_The structure used is mentioned  
typedef struct {  
    // symbol registry Symbol \*symbols;  
    int numSymbols;  
    int capSymbols;  
    // productions (one entry per LHS nonterminal that appears in grammar.txt)  
    Production \*prods;  
    int numProds;  
    int capProds;  
    int startSymbol;   // LHS of first production read  
    // quick counts (optional but handy)  
    int numTerminals;  
    int numNonTerminals; } Grammar; \_\_\_\_\_
- ii. FIRST and FOLLOW sets computation automated (yes /no)\_\_\_\_ Yes \_\_\_\_\_
- iii. Name the functions (if automated) for computation of First and Follow sets \_\_\_\_computeFirst(), computeFollow() in defined in file firstfollow.c \_\_\_\_\_
- iv. If computed First and Follow sets manually and represented in file/function (name that) \_\_\_\_\_

[D].Error Handling

- v. Attempted (yes/ no): Yes
- vi. Describe the types of errors handled \_\_\_\_ We handle predictive parsing syntax errors using panic-mode recovery with SYNCH entries in the parse table \_\_\_\_\_

## 7. Compilation Details:

[A].Makefile works (yes/no): YES

[B].Code Compiles (yes/ no): YES

[C]. Mention the .c files that do not compile: NONE

[D]. Any specific function that does not compile: NONE

[E]. Ensured the compatibility of your code with the specified gcc version (yes/no): YES

8. Driver Details: Does it take care of the options specified earlier(yes/no): YES

9. Execution

[A]. status (describe in maximum 2 lines): \_\_\_\_ all files can be compiled using make and executed as per instructions given \_\_\_\_\_

[B]. Gives segmentation fault with any of the test cases (1-6) uploaded on the course page. If yes, specify the testcase file name: \_\_\_\_\_ No \_\_\_\_\_

10. Specify the language features your lexer or parser is not able to handle (in maximum one line) None

11. Are you availing the lifeline (Yes/No): No

Declaration: We, Arjun Neekhra, Arpita Tomar, Arvind Annamalai Balasubramanian, Mehul Srivastava, S Pranav Kumar, Tarun G declare that we have put our genuine efforts in creating the compiler project code and have submitted the code developed only by us. We have not copied any piece of code from any source. If our code is found plagiarized in any form or degree, we understand that a disciplinary action as per the institute rules will be taken against all of us in our team and we will accept the penalty as decided by the department of Computer Science and Information Systems, BITS, Pilani.

Name: Arjun Neekhra

ID: 2022B3A71033P

Name: Arpita Tomar

ID: 2022B4A70596P

Name: Arvind Annamalai Balasubramanian

ID: 2022B4A70581P

Name: Mehul Srivastava

ID: 2022B3A70604P

Name: S Pranav Kumar

ID: 2022B2A71101P

Name: Tarun G

ID: 2022B3A70453P

Date: 02/03/2026

-----  
*Not to exceed 3 pages.*